

BigFix Capacity Planning: Core Platform

an HCL Product

HCLSoftware

Document version 11.x.13

Mark Leitch
BigFix Platform, Performance & Secure Engineering

© Copyright 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026 HCL Technologies Ltd. HCL Technologies Ltd., and the HCL Technologies Ltd. logo are trademarks of HCL Technologies Ltd., registered in many jurisdictions worldwide.

CONTENTS

CONTENTS **3**
LIST OF FIGURES **5**
REVISION HISTORY **6**
THANKS **7**
1 INTRODUCTION **8**
2 BIGFIX OVERVIEW **9**
2.1 FUNCTIONAL OVERVIEW 9
2.2 ARCHITECTURAL OVERVIEW 10
3 CAPACITY PLANNING METHODOLOGY **12**
3.1 UNITS OF CAPACITY PLANNING 12
3.1.1 Cores, CPUs, and vCPUs *12*
3.1.2 Memory *12*
3.1.3 Network *12*
3.1.4 Storage *12*
3.1.5 Cloud vs SotP vs SotA *13*
3.1.6 Concurrent Users *13*
3.2 CAPACITY PLANNING TOOLS 13
4 CAPACITY PLANNING **15**
4.1 THE EVOLUTION OF BIGFIX CAPACITY PLANNING 15
4.2 THE ROOT SERVER 16
4.3 THE WEB REPORTS AND BES EXPLORER SERVICES 17
4.4 THE WEBUI SERVER 17
4.5 THE BIGFIX UI SERVER 18
4.6 THE CONSOLE 19
4.7 THE RELAYS AND ENDPOINT INFRASTRUCTURE 20
4.7.1 Relay Infrastructure Requirements *20*
4.7.1.1 CPU Requirements 20
4.7.1.2 Memory Requirements 21
4.7.1.3 Storage Requirements 21
4.7.1.4 Network Requirements 21
4.7.2 Top Level Relays *22*
4.7.3 Leaf Node Relays *22*
4.7.4 Persistent Connection Support *23*
4.7.5 The Archive & Upload Manager *23*
4.7.6 Relay Virtualization *24*
4.8 THE PLUGIN PORTAL 24
4.9 BIGFIX MODERN CLIENT MANAGEMENT (MCM) 25
4.10 BIGFIX REMOTE CONTROL (RC) 26
4.11 BIGFIX OPERATING SYSTEM DEPLOYMENT (OSD) 27
4.12 BIGFIX SERVER AUTOMATION (SA) 29
4.13 BIGFIX COMPLIANCE ANALYTICS 30
4.14 MESSAGE LEVEL ENCRYPTION (MLE) 31
5 PERFORMANCE MANAGEMENT **32**
5.1 PERFORMANCE MANAGEMENT APPROACHES 32
5.1.1 Performance vs Ease of Management *32*
5.1.2 The Storage Revolution *32*
5.1.3 When to Throw Hardware at the Problem *32*
5.1.4 Collocation and Anti-collocation Options *33*
5.2 BIGFIX MANAGEMENT RECOMMENDATIONS 33

5.2.1 BigFix FillDB Configuration 33
5.2.2 BigFix Relay Configuration 34
5.2.3 Agent Heartbeat Frequency 34
5.2.4 Agent Multicore Exploitation 35
5.2.5 Console Refresh Frequency 35
5.2.6 Data Archiving 35
5.2.7 WebUI 36
5.2.8 The Plugin Portal 36
5.2.9 The MDM Docker Host 37
5.3 MS SQL MANAGEMENT RECOMMENDATIONS 37
5.3.1 MS SQL Parallelism 37
5.3.2 MS SQL Query Performance 38
5.3.3 MS SQL Always On Availability Groups for Read Scale 38
5.3.4 MS SQL Cloud Services 39
5.3.5 MS SQL Secure Settings 39
5.3.6 MS SQL Transparent Data Encryption 39
5.4 DB2 MANAGEMENT RECOMMENDATIONS 40
5.4.1 DB2 Instance Configuration 40
5.4.2 DB2 Query Performance 40
5.5 OPERATING SYSTEM MANAGEMENT RECOMMENDATIONS 40
5.5.1 Windows TCP/IP Port Management 41
5.5.2 Linux Swappiness 41
5.5.3 Linux ulimit Management 41
5.5.4 Virtualization Management 41
5.5.5 "Right Sizing" the CPU Allocation 42
5.5.6 VMware Snapshot Management 42
5.5.7 VMware Latency Management 42
5.5.8 Virtual IO Management 42
5.5.9 The Linux IO Scheduler 43
REFERENCES 44
NOTICES 45
TRADEMARKS 46

LIST OF FIGURES

FIGURE 1: REVISION HISTORY 6
FIGURE 2: BIGFIX ARCHITECTURE 10
FIGURE 3: CAPACITY PLANNING TOOL INVOCATION EXAMPLE 14
FIGURE 4: ROOT SERVER CAPACITY PLANNING REQUIREMENTS 17
FIGURE 5: WEB REPORTS AND BES EXPLORER CAPACITY PLANNING REQUIREMENTS 17
FIGURE 6: WEBUI CAPACITY PLANNING REQUIREMENTS 18
FIGURE 7: BIGFIX UI CAPACITY PLANNING REQUIREMENTS 18
FIGURE 8: CONSOLE CAPACITY PLANNING REQUIREMENTS 19
FIGURE 9: TERMINAL SERVER CAPACITY PLANNING REQUIREMENTS 19
FIGURE 10: BIGFIX RELAY INFRASTRUCTURE 20
FIGURE 11: TOP LEVEL RELAY CAPACITY PLANNING REQUIREMENTS 22
FIGURE 12: LEAF NODE RELAY SCALABILITY 22
FIGURE 13: LEAF NODE RELAY CAPACITY PLANNING REQUIREMENTS 23
FIGURE 14: PERSISTENT CONNECTION LIMITS 23
FIGURE 15: ARCHIVE & UPLOAD MANAGER CONFIGURATION 23
FIGURE 16: BIGFIX RELAY VIRTUALIZATION PERFORMANCE 24
FIGURE 17: PLUGIN PORTAL CAPACITY PLANNING REQUIREMENTS 25
FIGURE 18: BIGFIX REMOTE CONTROL INFRASTRUCTURE 26
FIGURE 19: BIGFIX REMOTE CONTROL SERVER CAPACITY PLANNING REQUIREMENTS 27
FIGURE 20: BIGFIX REMOTE CONTROL BROKER CAPACITY PLANNING REQUIREMENTS 27
FIGURE 21: BIGFIX REMOTE CONTROL CONTROLLER/TARGET/GATEWAY CAPACITY PLANNING REQUIREMENTS 27
FIGURE 22: BIGFIX SERVER AUTOMATION INFRASTRUCTURE 29
FIGURE 23: BIGFIX SERVER AUTOMATION PLAN ENGINE CAPACITY PLANNING REQUIREMENTS 29
FIGURE 24: BIGFIX COMPLIANCE ANALYTICS CAPACITY PLANNING REQUIREMENTS 30
FIGURE 25: MESSAGE LEVEL ENCRYPTION CAPACITY PLANNING REQUIREMENTS 31
FIGURE 26: FILLDB PARALLELISM IMPACT 34
FIGURE 27: MS SQL QUERY PERFORMANCE PACKAGE CACHE 38
FIGURE 28: DB2 INSTANCE CONFIGURATION 40
FIGURE 29: DB2 QUERY PERFORMANCE SNAPSHOT 40
FIGURE 30: IO BENCHMARK WORKLOAD PROFILES 43
FIGURE 31: MODIFYING THE LINUX IO SCHEDULER 43

REVISION HISTORY

Date	Version	Revised By	Comments
March 31st, 2020	10.x.1	MDL	Initial distribution based on 9.x.18.
Changes in the v10 publication stream deleted for brevity.			
October 1st, 2023	11.x.1	MDL	BigFix 11 revision.
November 26th, 2023	11.x.2	MDL	Added MS SQL AG information.
March 25th, 2024	11.x.3	MDL	BES Explorer capacity planning.
June 1st, 2024	11.x.4	MDL	MS SQL Cloud Services addition.
July 11th, 2024	11.x.5	MDL	Remove MS SQL AG constraint.
March 31st, 2025	11.x.6	MDL	BigFix 11.0.4 update.
April 22nd, 2025	11.x.7	MDL	Added Remote Control & Server Automation.
June 15th, 2025	11.x.8	MDL	Added MS SQL TDE reference.
November 10th, 2025	11.x.9	MDL	Added BigFix OSD reference.
March 25th, 2026	11.x.10	MDL	Added BigFix Compliance.
August 1st, 2026	11.x.11	MDL	BigFix 11.0.7 content.
October 5th, 2026	11.x.12	MDL	Various refinements.
October 31st, 2026	11.x.13	MDL	BigFix UI additions.

Figure 1: Revision History

THANKS

Thanks to the BigFix development team for their continued excellence and for producing such a well performing and useful product. Thanks also go out to the incredible BigFix customers, who have helped to make the product what it is today and have provided feedback to improve this guide.

Special thanks to the BigFix performance team. Their hard work has made BigFix scale higher, go faster, and work better than ever. Notable members of the performance team include the following (in alphabetical order).

- Antonio Mangiacotti
- Donatella Girolamo
- Indu Sedagam
- Laxmi Ramesh Mallur
- Massimo Marra
- Paolo Cavazza

1 Introduction

Capacity planning involves the specification of the various components of an installation to meet customer requirements, often with growth or timeline considerations. BigFix offers endpoint lifecycle and security management for large scale, distributed deployments of servers, desktops, laptops, and mobile devices across physical and virtual environments.

This document will provide an overview of capacity planning for the BigFix Version 11.x solution. In addition, it will offer management best practices to achieve a well performing installation that demonstrates service stability. This will include the deployment of the BigFix management servers into cloud, or virtual, environments. Capacity planning for virtual environments typically involves the specification of sufficient physical resources to provide the illusion of unconstrained resources in an environment that may be characterized by highly variable demand

In this document we will provide a BigFix overview, including functionality and architecture. We will then offer the capacity planning recommendations, built on a set of base definitions. Finally, configuration recommendations will be provided for achieving a highly functional and performant installation.

Note: This document supersedes all previous BigFix capacity planning documentation. In addition, while the previous capacity planning reference from the author spanned several topics with a prose-based approach, this document is more specific with a bullet point approach. The rationale for this is to have smaller, more easily digestible documents with easily understood recommendations.

As a result, the following documents are considered to offer a suite of performance, capacity, and maintenance reference information for BigFix. See the References section for relevant URLs.

- BigFix Capacity Planning (this document).
- BigFix Non-Functional Requirements: A Checklist Approach
  A set of guidelines and recommendations for BigFix performance and security.
- BigFix Maintenance Guide.
  A set of configuration and maintenance recommendations for BigFix.
- MX Performance Toolkit for BigFix.
  A set of tools and performance management approaches for BigFix.

Note: This document is considered a work in progress. Capacity planning recommendations will be refined and updated as new BigFix releases are available. While the paper in general is considered suitable for all BigFix Version 10.x and 11.x releases, with new content as noted, it is best oriented towards BigFix Version 11.0.0 onwards. In addition, several references are provided in the References section. These papers are highly recommended for readers who want detailed knowledge of BigFix server configuration, architecture, and capacity planning.

2 BigFix Overview

An overview of BigFix will be provided from the following perspectives:

1. Functional.
2. Architectural.

2.1 Functional Overview

The BigFix portfolio provides a comprehensive security solution encompassing several operational areas. These areas include the following.

- Lifecycle management (asset discovery and inventory, software distribution, patch management, operating system deployment, remote control).
- Security and compliance (security configuration management, vulnerability management, patch management, anti-virus and anti-malware client management, network self-quarantine).
- Patch management.
- Power management.
- Software use analysis.
- Core protection.
- Server automation.

In general, BigFix spans the broadest OS and device set in the industry, including the management of physical and virtual servers, PCs, Macs, tablets, smartphones, embedded and hardened devices, and point of sale devices. This is managed via a scalable distributed infrastructure that includes a lightweight dedicated agent. We will describe this infrastructure in the architectural overview section.

2.2 Architectural Overview

The following diagram provided a basic view of the BigFix architecture.

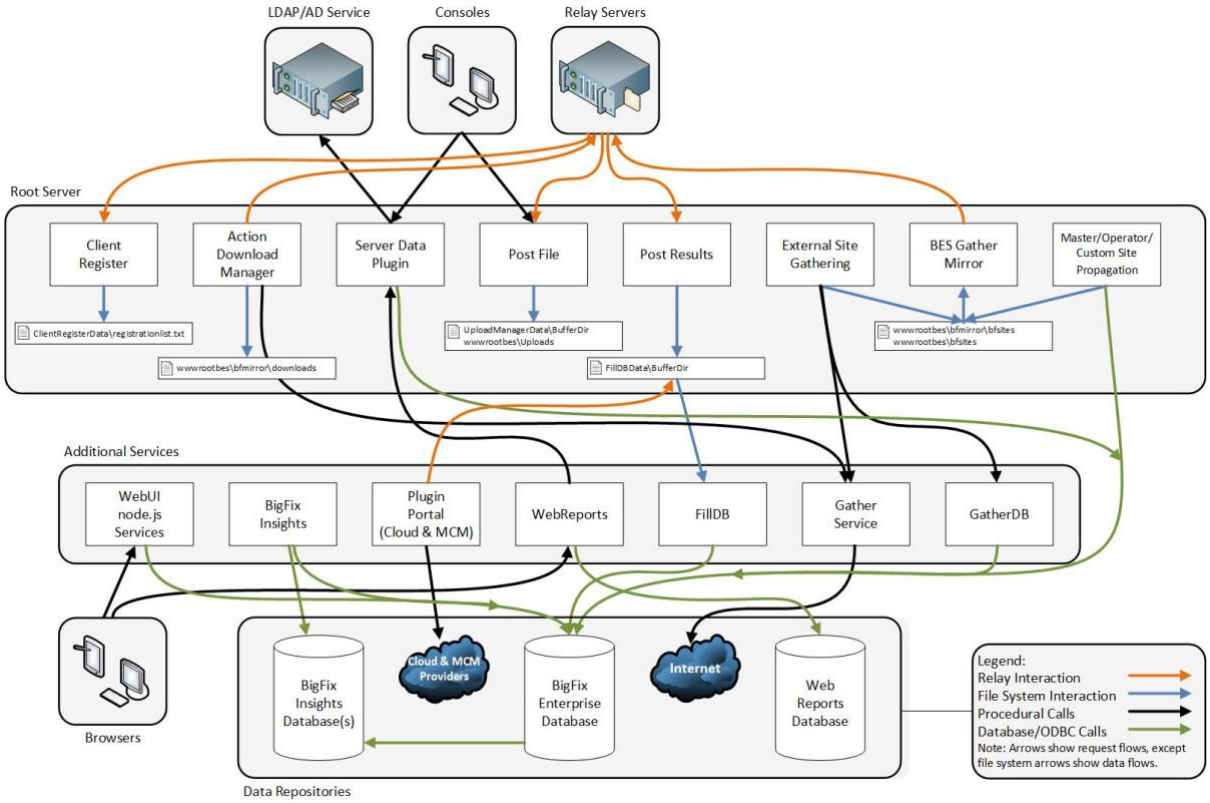


Figure 2: BigFix Architecture

The notable components of this diagram follow.

- Root Server.
  The base BigFix Enterprise Server. It is comprised of a set of core services as identified.
- Console.
  A management console (user interface) for BigFix operators. The console is a Windows only application. A console server is used to support one or more instances of the BigFix Console.
- The WebUI.
  A Node.js instance with an associated database intended to support the Web based user interface.
- The BigFix UI (not shown).
  A new browser-based UI intended to be the follow-up for WebUI.
- The Web Reports and BES Explorer Services.
  The Web Reports service can provide a variety of stock and custom reports for one or more BigFix server installations. With BigFix version 11.0.2, the Web Reports service has been refactored with the introduction of the BES Explorer service. A future version of this document will revise the architecture diagram.
- Relays.
  A distributed, hierarchical infrastructure to manage the deployment of BigFix agents across diverse network topologies.
- Plugin Portal (new for BigFix 10).
  The Plugin Portal provides the common server infrastructure for the Cloud and Modern Client Management (MCM) technologies delivered in BigFix 10. The initial cloud offering provides support for VMware, Azure, and Amazon Web Services (AWS) deployments. The MCM

capability applies to Android, Windows, iPhone, iPad, and Mac deployments.

Note: The full MCM stack is not shown in the architecture figure. The MCM capacity planning section below offers further details on the MCM deployment topology.

- BigFix Insights (new for BigFix 10).
  BigFix Insights offers new reporting capability, typically based on a standalone data replica to isolate its workload from the BigFix root server. It is accompanied by new reporting content in the WebUI, to improve the visualization and understanding of BigFix deployments.
- Agents (as part of the client population).
  A lightweight, native agent that manages the endpoint.
- Content (Fix or Fixlet Servers, represented via the Internet content).
  These servers are used as the object repository for all client content (Fixes, Fixlets, tasks, baselines, and analyses). In addition, dashboards, wizards, and WebUI applications are delivered via the servers. Content is utilized by the agent to determine relevance, compliance, and remediation activities.
- Disaster Server Architecture (aka DSA, not shown).
  The DSA is a server replication architecture intended to provide fault tolerance.
- The Database Management Server, or DBMS (either Microsoft SQL Server or IBM DB2 for Linux, UNIX, and Windows, also referred to as DB2 LUW).

In general, anti-collocation is possible for the core BigFix server components. For example, the root server, Web Reports, BES Explorer, WebUI, and DBMS may all be distinct physical or virtual server instances. The pros and cons of anti-collocation are described later in this document.

3 Capacity Planning Methodology

The capacity planning methodology will be broken down into the following topics.

1. Units of capacity planning.
2. Capacity planning tools.

3.1 Units of Capacity Planning

To properly manage capacity, some base definitions must be understood. Adhering to these definitions is critical to the accuracy of the results. We will break the units down into several physical, virtual, and organizational components.

3.1.1 Cores, CPUs, and vCPUs

Today's multiprocessor architectures are often defined in terms of cores and may be physical cores or virtual CPUs (aka vCPUs). The following rules of thumb apply for our capacity planning outlook.

- In terms of pure clock speed, per typical cloud definitions, we will generally consider a CPU core to be a relatively current generation 2.0+ GHz/core implementation.
- A discrete core and a vCPU can be considered equivalent. However, more management is typically required for a vCPU due to the hypervisor scheduler and contention for vCPUs due to the virtual configuration and over commitment policy. As a result, hypervisor monitoring is required to ensure a vCPU is operating in the range of 90+% of the capability of a physical core.
- A hyper threaded core does not have the throughput capability of a "pure" core and can be considered to have on the order of 30% of the capability of a pure core.
- Our sizing approach is based on pure, physical cores at 2.0+ GHz/core. For virtual or hyper threaded cores, the above efficiency ratios should be part of the sizing methodology.

3.1.2 Memory

- Modern systems are extremely good at exploiting memory. If you are able to exceed the memory requirements, the operating system will use the additional memory to improve caching and throughput.
- On modern systems, it is not unusual to see high memory utilization approaching 100% of physical memory. This is often quite healthy, as long as the system is not paging significantly.
- Database servers in particular like to exploit memory, and if extra memory is available, the first priority should be to allocate it to the database server.

3.1.3 Network

- High bandwidth and low latency memory is readily available and should be used wherever possible in the data center. For example, 10 gigabit ethernet and teamed adapters are common and will offer excellent network capability.
- Beyond the data center, mapping the network for latency, bandwidth, etc. is critical for relay and endpoint management.

3.1.4 Storage

- The BigFix storage requirement is for storage to offer in excess of 5,000 IOPS (IO Operations per Second) with 1ms latency. This is especially critical for the database server volumes.

- This capability is easily achieved with local SSD with an AHCI interface.
- This capability can be massively over-achieved with flash-based storage appliance and local NVME based flash storage. For example, on benchmark systems we can manage in excess of 100,000 IOPS with 0.1ms latency with a single NVME flash-based drive. See the References section for further information on NVME and the “storage revolution”.

3.1.5 Cloud vs SotP vs SotA

We will provide a few definitions for server deployments.

- Cloud: A public or private instance with:
  CPU: 2.0 Ghz or better
  IO: 5,000 IOPS with 1ms latency or better.
- SotP: A “State of the Practice” environment. This is an affordable present-day server with:
  CPU: 3.0 Ghz or better.
  IO: 100,000 IOPS with 1ms latency or better.
- SotA: A “State of the Art” deployment. The definition is in continuous upheaval due to the array of continuous improvements for server deployments.

In general, SotP deployments are recommended for their combination of affordability, economy, and performance.

3.1.6 Concurrent Users

For the user interface component, the number of concurrent users that need to be supported is important. It is also highly misunderstood. The following definition for concurrent users should be adhered to for the WebUI instance.

- For the concurrent user population, consider:
  P = total population for an instance (including administrators, end users, etc.).
  C = the concurrent user population for an instance of BigFix.
- Concurrent users (C) are the set of users within the overall population (P) that are actively managing the environment at a point in time (e.g., administrator operations in the user interface, endpoint operations, etc.).
- In general, P is a much larger value than C (i.e., P >> C). For example, it is not unrealistic that a total population of 200 users may have a concurrent user population of 20 users (i.e., 10%).
- The number of concurrent users supported by a solution is a function of request response times under load. Numbers are driven by lab and field-based performance evaluations that consider the user experience (response times), server load, and server impact.
- Lab based simulations use aggressive think times. For example, user response times in the interval [1s,10s] are used. As a result, when mapping an organization’s user population to the number of concurrent users, the standard is for actively engaged users driving high request rates.

3.2 Capacity Planning Tools

Several complex capacity planning tables will be provided. These tables constitute the reference content for BigFix. However, a set of performance and capacity planning tools are provided to greatly simplify the capacity planning experience. See the MX Performance Toolkit in the References section for obtaining the distribution and associated documentation.

The following figure shows an example of invoking the capacity planning tool. The options are intended to be self-explanatory, with full descriptions provided in the reference document.

```
MXCapacity –-endpoints 60000 –-concusers 10 –-service root dbms webui webreports relays
```

Figure 3: Capacity Planning Tool Invocation Example

4 Capacity Planning

The capacity planning recommendations will be broken down across the following components.

- The root server, including the database server.
- The WebUI server.
- The BigFix UI server.
- The console.
- The relays and endpoint infrastructure.
- The Plugin Portal.
- Modern Client Management (MCM).
- Remote Control (RC).
- Operating System Deployment (OSD).
- Server Automation (SA).
- Compliance.
- Message Level Encryption (MLE).
- Note: Prior versions of this guide provided capacity planning recommendations for BigFix Insights. The recommendations are now part of the core product documentation: URL.

Some considerations should be kept in mind for the recommendations.

- Capacity planning recommendations are general purpose for a “typical” workload. We model the “typical” workload in our performance labs and consolidate field results.
- Capacity planning recommendations are provided in terms of the number of managed endpoints. This is a simplification to make the recommendations consumable, but there are many more dimensions that may apply for a specific installation.
- In the event you are within a range of capacity planning recommendations (e.g., somewhere between 50,000 and 100,000 managed endpoints for the BigFix root server), you may start at the low end of the range and grow, assuming your workload and system behavior is well understood. This applies to the CPU, memory, and storage allocations. The IO subsystem and network requirements are universal.
- Monitoring over time is always recommended. This guide includes specific references for how to monitor at the operating system, application, and database levels.
- Capacity planning is seldom static. Systems grow over time. Entropy increases. Maintenance operations are typically required, especially at the database level, to manage this and ensure performance stability. References for this are provided in this guide.

4.1 The Evolution of BigFix Capacity Planning

Historically, BigFix has supported a 250k agent deployment on a single root server. With BigFix 10.0.0, this was extended to 250k agents and 50k objects managed via the Plugin Portal.

With BigFix 10.0.4 through BigFix 11, we now support 300k for any set of devices, whether agent based or managed via the Plugin Portal. For example, the following deployments are all supported on a single root server.

- 300k agents.
- 200k agents with 100k MCM objects.
- 300k MCM objects (with no agents).

- 100k agents distributed amongst 100k cloud objects and 100k MCM objects.

The general support statement is 300k devices in the BFEnterrprise “Computers” table, that may or may not be correlated devices.

4.2 The Root Server

The following table describes the root server’s (aka the management server) capacity planning recommendations as a function of the scale of the “managed to” estate.

- The recommendations are for a collocated server configuration. Finer grained, non-collocated recommendations are available via the referenced capacity planning tools.
- Requirements for the root server include the base operating system requirements. All other capacity planning numbers are in addition to the base operating system requirements.
- Storage requirements:
    - It is advantageous to have distinct storage subsystems/controllers for specific components. For example, a recommended best practice is for discrete storage subsystems for each of the base system, the database logs, and the database containers.
    - The database logs are typically characterized by the need for very fast sequential IO.
    - The database containers can have much more diverse access patterns and are highly insulated from storage impact by virtue of the database buffer pools.
    - A reasonable storage breakdown is 30% for the base, 5% for the database logs, and the remainder (65%) for the database containers.
    - A good rule of thumb is the core BigFix Enterprise database grows by 5GB for every 10,000 computers added. The overall storage impact will be higher than this (logs, buffers, temps, backups, etc.).
    - Suitable storage is recommended to accommodate the growing database size and associated management overhead (e.g., a working set of database backups). The storage requirements reflect the requirements for BigFix 9.5 onwards, where the Unicode base has driven greater storage requirements.
    - The BigFix application directory also contains a download cache which defaults to 1GB. We recommend increasing this cache size to somewhere between 100GB and 1TB (or higher) depending on needs. The storage requirements below should be increased to account for the desired cache size.
- Network requirements: A 1 Gbps network or better for the management server infrastructure.

Agent Deployment Size	CPU	Memory (GB)	Storage (GB)
< 1,000	4[1]	16	100
10,000	4	16	250
50,000	6	24	300
100,000	12	48	500
150,000	16	72	750
200,000	20	96	1,000
250,000	24	120	1,250
300,000	28	144	1,500

Figure 4: Root Server Capacity Planning Requirements

4.3 The Web Reports and BES Explorer Services

The Web Reports service is assumed to be collocated with the root server instance. The BES Explorer service, introduced in BigFix 11.0.2, must always be anti-collocated with the root server instance. Their capacity planning requirements are similar given the Explorer is an advanced form of Web Reports, and there are options for anti-collocation for Web Reports as well. Both services may be deployed in a horizontal cluster configuration, where multiple service instances are utilized.

For both services, the following capacity planning recommendations may be used when anti-collocated with the root server. The “plus sizing” indicates these resources are in addition to the base deployment node requirements (e.g., OS and other applications). The MAX function means take the maximum of the two values in parentheses. Note the additional storage requirements are only for Web Reports as BES Explorer does not have a dedicated database or cache. As always, monitoring is required as utilization will be highly dependent on reporting and application workloads, as well as request parallelism.

Component	Additional CPU	Additional Memory (GB)	Additional Storage (GB) (Web Reports Only)
Web Reports/ BES Explorer	MAX(2, 10% of Root Server CPU)	MAX(4, 15% of Root Server Memory)	MAX(10, 5% of Root Server Storage)

Figure 5: Web Reports and BES Explorer Capacity Planning Requirements

4.4 The WebUI Server

The BigFix WebUI offers a scalable and highly responsive management interface. There have been several iterations of the WebUI server. This document will describe the results for the BigFix March 2020 update (typically 10.0 based) based WebUI server. If you are on a prior version of the server, an upgrade is strongly recommended as significant improvements have been delivered providing improved scale, function, and user experience.

[1] While more granular sizing is possible here, to ensure base operating system health and given the commodity level of a system with four processor cores, this is considered a reasonable and minimal base.

- The recommendations are in addition to the root server requirements.
- If an anti-collocated instance is deployed (meaning an instance not deployed on the root server), the CPU requirements should be split across the database and WebUI servers, and the storage should be added to the database server.
- In terms of recommended scalability limits, both the Windows and Linux WebUI instances support 36 concurrent users on a 300k device deployment base. Once again, these are highly active concurrent users per the previously provided capacity planning definitions.
- Concurrent users would typically be non-master operators, managing a subset of the estate.
- It is possible to manage at a larger scale based on user operations, infrastructure capability, etc. However, the stated bounds should be considered a good “rule of thumb” for the scale of the solution.

Component	Additional CPU	Additional Memory (GB)	Additional Storage (GB)
BigFix WebUI	+2 per 10 concurrent users	+2 per 10 concurrent users	15% of BigFix database

Figure 6: WebUI Capacity Planning Requirements

4.5 The BigFix UI Server

The BigFix UI, introduced with BigFix 11.0.7, is a replacement for the WebUI. The BigFix UI Server has the following capacity planning considerations.

- The recommendations are in addition to the root server requirements, including BES Explorer.
- The capacity planning elements are broken down into the new BigFix UI service, and the existing BES Explorer and DBMS services.
- For the DBMS, the assumption is the resources are added to an existing instance. If a dedicated BigFix UI SQL 2025 instance is deployed, add the resources to both BFEnterrprise and also add to a 4 vCPU and 8GB base BigFix UI SQL 2025 instance.
- In terms of recommended scalability limits, the BigFix UI supports up to 36 concurrent users on a 300k device deployment base. Once again, these are highly active concurrent users per the previously provided capacity planning definitions.
- Concurrent users would typically be non-master operators, managing a subset of the estate.
- The number of concurrent users and their associated sessions is logged by the BigFix UI. The logging interval is configurable in the AppConfig.yaml file. By default, it is set to 5 minutes. The configuration parameter used to define the interval is “BESUserCompositionPollingIntervalInMin”.
- It is possible to manage at a larger scale based on user operations, infrastructure capability, etc. However, the stated bounds should be considered a good “rule of thumb” for the scale of the solution.

Component	Additional CPU per 36 CUs	Additional Memory (GB) per 36 CUs	Additional Storage (GB)
BigFix UI	+2	+2	+50
BES Explorer	+4	+4	+50
DBMS	+2	+2	10% of BigFix database

Figure 7: BigFix UI Capacity Planning Requirements

4.6 The Console

The following table shows capacity planning requirements for a workstation-based console.

- Expectation is data center level network speeds are available.
- The site cache is expected to be on the order of 20MB per external site.
- The question often arises how many console operators may be supported. A primary selling feature is the ability of a small number of operators to manage a large estate. However, if fine grained management is required, a base of 300 operators may be managed, with up to 50 concurrent operators at any point of time. This level of scale requires careful attention to the console infrastructure. Proceeding beyond these values would require understanding of the infrastructure and associated workload impact.

Deployment Size	CPU	Memory (GB)	Storage (GB)
< 10,000	1	2	0.25 + site cache
< 100,000	2	4	2GB + site cache
> 100,000	2	6	4GB + site cache
> 200,000	2	8	8GB + site cache

Figure 8: Console Capacity Planning Requirements

The following table shows capacity planning requirements for a terminal or Citrix server-based implementation.

- The expectation is data center level network speeds are available for the server, and each server may be managing on the order of 10-20 concurrent users (remote users, meaning they may not reside in the data center).
- In the event a greater number of concurrent users are in effect, the general rule of thumb is to add on the order of 1 CPU and 2-6 GB of RAM for every additional concurrent user (RAM is dependent on the deployment size).
- As always, requirements are workload dependent so monitoring of the system under load is always recommended. Ranges are given where appropriate with the expectation that monitoring may be used to fine tune in the customer environment.

Deployment Size	CPU	Memory (GB)	Storage (GB)
< 10,000	8 - 16	8 - 16	20
< 100,000	10 - 20	16 - 48	80GB
> 100,000	10 - 20	32 - 80	80GB - 240GB

Figure 9: Terminal Server Capacity Planning Requirements

4.7 The Relays and Endpoint Infrastructure

The following diagram provides a deployment scenario for the BigFix relay and agent infrastructure[2].

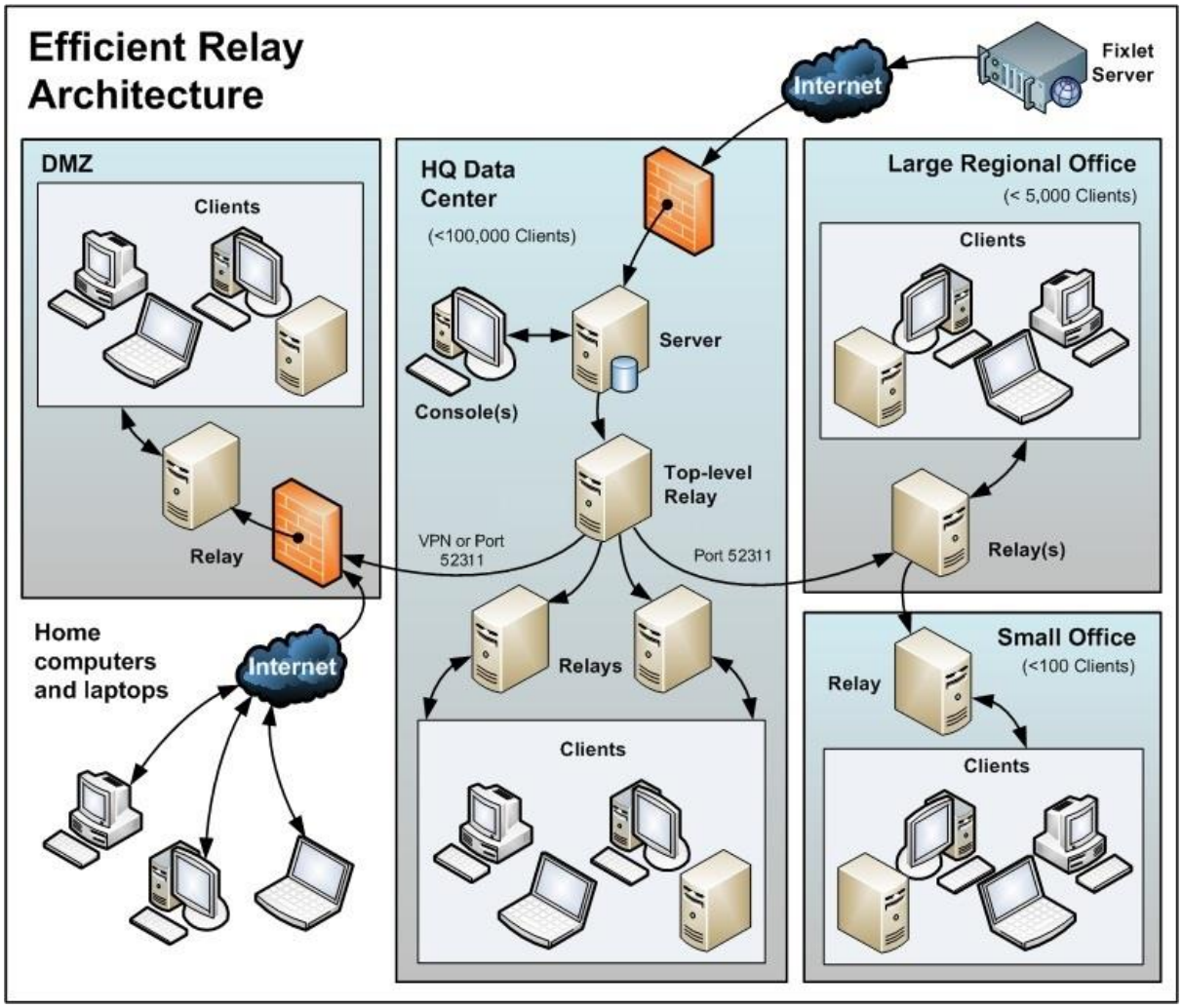


Figure 10: BigFix Relay Infrastructure

The relays are typically deployed in a hierarchy with top level relays serving other relays, and the leaf node relays (often referred to as site level relays) serving the endpoints. In terms of capacity planning, we will describe the infrastructure requirements and then break down the discussion in terms of top level relays and leaf node relays.

4.7.1 Relay Infrastructure Requirements

We may break down the infrastructure requirements into CPU, memory, network, and storage.

4.7.1.1 CPU Requirements

- For the CPU specification, we will continue with the standard vCPU configuration, and provide a pair of [minimum,recommended] vCPU core values.
- BigFix can run successfully with the minimum configuration. However, in the event additional resources are available, or deploying into a virtual environment where over commitment is possible, and resources will be dynamically utilized, the recommended values will permit

[2] Diagram courtesy of the BigFix Product Documentation: URL.

greater concurrency and reduce processor latency.

4.7.1.2 Memory Requirements

- Similarly, the memory requirements consist of a [minimum,recommended] pair. The minimum values may be deployed successfully, but the recommended values will permit improved operating system memory management and buffering.

4.7.1.3 Storage Requirements

- Storage requirements are subject to configuration of the relay. The following calculation is representative of general relay storage requirements.

  Storage in GB = OS + 3GB (BigFix binaries + logs) +
  2GB (Default Cache) + Extended Cache (if additional cache space is configured) +
  300 MB/site (site cache for relay)

- For the relay storage requirements, the operating system storage may vary widely. For example, Windows 10 64 bit requires 20GB. Something like the Linux Tiny Core, where relays have been successfully demonstrated, requires on the order of megabytes. The net is we have high performance Windows relays running with 25GB, and they have been proven to virtualize extremely well.
- For Linux relays it can be important to manage the available file system handles, especially for large deployments. For example, the ulimit “nofiles” configuration parameter should be set to a minimum of 16384 for releases up to 11.0.6, and a minimum of 32,768 for 11.0.7 onwards. The higher number for 11.0.7 addresses the increased capability for persistent connections that will consume file handles. See section 5.5.3 for more information on managing ulimit.

4.7.1.4 Network Requirements

- Network requirements depend heavily on the deployment, including available bandwidth and the possibility for dynamic network shaping.
- From a relay perspective, to push a 10MB download to 5,000 endpoints can result in on the order of 50GB of network traffic depending on the relay topology. For such a distribution, it is important to determine if the relay throttling configuration needs to be managed, or the download controlled and distributed over a specific timeline.
- In terms of base relay health, obviously the more available network bandwidth the better, but it is possible to deploy BigFix relays in highly constrained environments (e.g., < 10 Mbps) and manage successfully.
- A more difficult decision to reach is actual placement of the relays, and particularly the hierarchy of nodes. Decision points include network bandwidth, network latency, firewalls, server infrastructure, etc. Network topology maps typically exist for most enterprises. However, these maps rarely contain accurate metrics for network performance between nodes. Even when metrics are provided, they are often out of date or represent ideal conditions. In addition, network shaping often applies, meaning network characteristics may be dynamic based upon load.
- To facilitate understanding of the network and placement considerations, basic network ping tests may be performed. Sample ping commands follow.
  Basic ping: ping –c 10 <ip>
  Flood ping: ping –f –c 100000 –s 1500 –l 4 <ip>
  - Using these commands, a map may be built showing latency and packet loss.
  - Additional tests to demonstrate the number of hops via trace route (e.g., tracert) or equivalent are recommended. Secure copy (e.g., scp) tests for sample payloads are also helpful. In essence, once the network characteristics are defined, placement decisions typically become very straightforward.

- In lab tests, a general rule of thumb for a leaf relay is for every 1,000 endpoints served, bandwidth requirements of 100Mbps are typical.
    - This implies for 10,000 endpoints served by a leaf relay, 1Gbps or greater bandwidth is required.
    - Bandwidth is highly dependent on configuration and workload. Monitoring is recommended.
- See section 5.2.2 for further information on relay configuration management.

4.7.2 Top Level Relays

For the relays serving other relays (also known as top level relays), the following capacity planning recommendations apply.

- Top level relays are generally recommended once you approach 50,000 endpoints or over 100 relays (whichever comes first).
- A top level relay should manage no more than 100,000 endpoints or 120 relays (whichever comes first).
- The relays managing the endpoints offer low utilization and may possibly be collocated with server nodes already distributed in the enterprise.

Deployment Size (Endpoints Served)	CPU	Memory (GB)	Storage & Network
10,000	[1,2]	[2,4]	Per the infrastructure recommendations section.
25,000	[2,4]	[2,4]	
50,000	[3,6]	[3,6]	
75,000	[4,8]	[4,8]	
100,000	[5,10]	[5,10]	

Figure 11: Top Level Relay Capacity Planning Requirements

4.7.3 Leaf Node Relays

For the relays managing BigFix agents directly (aka the leaf relays), we will offer a range of scalability and configuration choices. Note the 11.0.7+ improvements are only available for the Windows, RHEL, and non-DMZ relays at the time of this writing.

BigFix Version	Non-Authenticating Relay Scale	Authenticating Relay Scale
< 11.0.4	5,000	5,000
11.0.4 through 11.0.6	10,000	5,000
11.0.7+ Windows/RHEL/Non-DMZ	20,000	20,000

Figure 12: Leaf Node Relay Scalability

In the event you would like a relay to serve somewhere between the specified ranges, you may use intermediate values in the ranges.

Deployment Size (Endpoints Served)	CPU	Memory (GB)	Storage & Network
1,000	[1,2]	[2,4]	Per the infrastructure recommendations section.
5,000	[2,4]	[4,6]	
10,000	[4,6]	[6,8]	
15,000	[6,8]	[8,10]	
20,000	[8,10]	[10,12]	

Figure 13: Leaf Node Relay Capacity Planning Requirements

4.7.4 Persistent Connection Support

Persistent connections are enabled between leaf relays and clients in the event that UDP notifications are not available. The maximum number of persistent connections has been elevated in BigFix 11.0.4 and 11.0.7 (see below). Once again, the 11.0.7+ improvements are only available for the Windows, RHEL, and non-DMZ relays at the time of this writing. For Linux based relays, the ulimit configuration is also important as described in section 5.5.3.

BigFix Version	_BESRelay_PersistentConnection_MaxNumber
BigFix 11.0.3 & Below	100
BigFix 11.0.4 to 11.0.6	1,000
BigFix 11.0.7 & Above Windows/RHEL/Non-DMZ	20,000

Figure 14: Persistent Connection Limits

4.7.5 The Archive & Upload Manager

The Archive & Upload Manager is a service that permits the client to upload files. In BigFix 11.0.4 an optimization may be enabled that avoids landing the file payload at each step in the relay chain. This optimization is not enabled by default. Internal benefits show a 600,000% improvement in payload efficiency, with fault tolerance added to the protocol to achieve much higher reliability. This also benefits applications such as BigFix Inventory.

BigFix Version	_BESRelay_PostFile_PassThrough
BigFix 11.0.3 & Below	n/a
BigFix 11.0.4 & Above	Enable

Figure 15: Archive & Upload Manager Configuration

4.7.6 Relay Virtualization

The BigFix relays virtualize extremely well. Their small system requirements and lightweight workloads are ideal for virtual machine deployments. In addition, it is possible to collocate several virtual relay instances in order to achieve virtual clusters for system throughput and redundancy.

- The following diagram shows the performance of an increasing number of relay requests, across several physical and virtual Linux instances.
- Performance across the instances is extremely consistent, providing excellent support for virtualizing relays.
- It should be pointed out that at the 50,000 request level, results are more inconsistent as we are at the infrastructure threshold for both virtual and physical deployments. For tuning virtual instances, the virtualization section later in this paper should be followed.

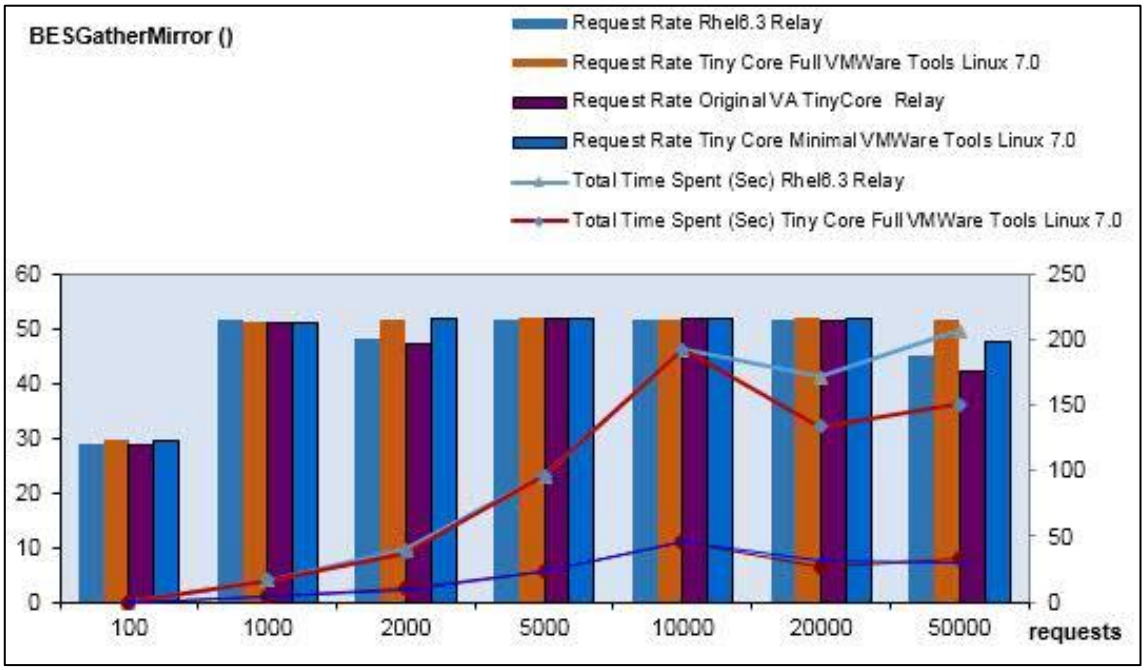


Figure 16: BigFix Relay Virtualization Performance

4.8 The Plugin Portal

The Plugin Portal offers a shift in BigFix capacity planning requirements.

- BigFix capacity planning has typically been in terms of the number of agents deployed. With the Plugin Portal and the associated Modern Client Management (MCM) and Cloud capability, there is now an agentless based scale component.
- In BigFix 10.0.4 through 11.0.6, each Plugin Portal instance may manage up to 75,000 cloud and MCM devices. With BigFix 11.0.7 this has been extended to 100,000 cloud devices.
    - For example, a Portal deployment for the MCM feature may manage 15k MacOS devices, 20k Windows devices, and 40k cloud devices for an aggregate 75k devices. Note prior to version 10.0.4, each Plugin Portal instance could support up to 10,000 compute devices.
- There may be several Plugin Portal instances, limited to an aggregate of 300k compute devices. For example, you may want to deploy multiple plugin portals geographically collocated with the devices they manage.
    - For the clustered Plugin Portal deployment, each portal instance should manage a distinct set of devices (i.e., disjoint sets of cloud instances, MDM servers, etc.). If

there is overlap, there is the potential for duplicate devices. To ensure the clarity and integrity of operations, it is not supported to have portals overlap device sets.

- For a full 300k deployment of cloud managed devices, a minimum of three (3) Plugin Portals should be deployed.

- The Plugin Portal may be deployed on a physical or virtual server. For virtual deployments, care should be taken around hypervisor efficiency.
- The capacity planning requirements are simplified. Capacity requirements are a function of the number of plugins and endpoints, as well as Fixlet content.
  - The capacity requirements are on top of the base operating system requirements for the compute instance. As always, upon deployment, runtime monitoring is recommended.
- The Plugin Portal may include two database instances: SQLite and MongoDB. While these are relatively small and highly specific database servers, it should be ensured that the server has reasonable IO capability. The recommendation is for the Portal to support a minimum IO throughput of 1,000 IOPS with < 1ms latency.
- In terms of deployment topologies for the Plugin Portal, there are many options.
  - A Portal instance may be deployed geographically remote from the BigFix root server. This may be desirable if there is a large, remote cloud instance that the portal is managing.
  - The Portal may generate large volumes of reports. As such, the higher-level relay infrastructure (if any) should offer high bandwidth. For example, having the Plugin Portal report to a top-level relay, false root relay, or the root server itself is recommended.

The following capacity planning requirements are based on the 10.0.8 and above versions. Support for 100k cloud devices was added in the 11.0.7 code stream. Note CPU requirements vary by class of Plugin (Cloud vs MCM). If both classes are active (i.e., Cloud and MCM plugins are enabled for a single portal instance), then the CPU requirements should be added.

Deployment Size	CPU	Memory (GB)	Storage (GB)
25,000	Cloud: +2 / Plugin MCM: +4	+4 per Plugin	+20
50,000	Cloud: +4 / Plugin MCM: +8	+6 per Plugin	+40
75,000	Cloud: +6 / Plugin MCM: +12	+8 per Plugin	+60
100,000 (11.0.7+, cloud devices only)	Cloud: +8 / Plugin MCM: n/a	+10 per Plugin	+80

Figure 17: Plugin Portal Capacity Planning Requirements

4.9 BigFix Modern Client Management (MCM)

The Modern Client Management (MCM) offering provides device management capabilities for Android, Windows, iPhone, iPad, and MacOS devices. Prior versions of this capacity planning guide provided capacity planning recommendations (through version 11.x.12). For the latest recommendations for the offering, it is recommended to contact product support.

4.10 BigFix Remote Control (RC)

Remote Control (RC) offers several deployment options with the following components.

- The RC Server.
  The overall orchestrator.
- The RC Controller.
  Forms a peer-to-peer session with the target.
- The RC Target.
  Forms a peer-to-peer session with the controller.
- The RC Gateway.
  Responsible for providing network management between the Controller and Target.
- The RC Broker.
  Permits operation over the Internet by bridging the Controller and Target communication.

The Gateway and Broker are only required if specific network topologies are in place (e.g., Internet based communication). See the following figure for a sample deployment architecture.

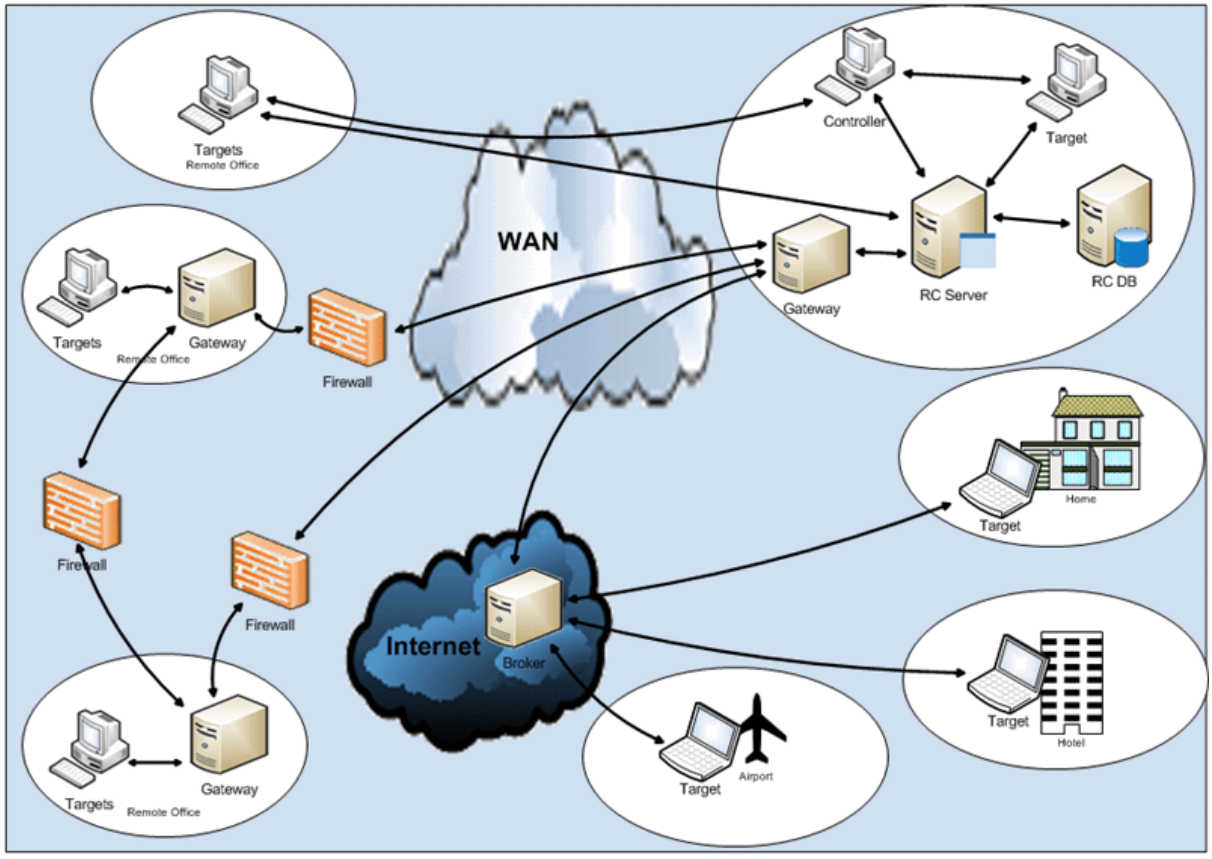


Figure 18: BigFix Remote Control Infrastructure

The Server and Broker capacity planning are a function of the number of targets managed. The remaining components are more universal in requirements. Detailed information is provided in the following tables, along with the following notes.

- Deployments should be 64 bit and meet or exceed the provided vCPU definition.
- The server storage requirement is on average two (2) MB per client in the database.
- Where noted, additional storage is required to maintain video sessions. For example, a five (5) minute eight (8) bit video will require 2MB of storage. Longer times and higher bit rates will require more storage.

Deployment Size (Targets)	CPU	Memory (GB)	Storage (GB)
5,000	+2	+4	+1
75,000	+4	+8	+4
225,000	+8	+16	+8

Figure 19: BigFix Remote Control Server Capacity Planning Requirements

Deployment Size (Targets)	CPU	Memory (GB)	Storage (GB)
On Demand	+1	+8	+2 + video sessions.
6,000	+2	+8	+2 + video sessions.
10,000	+4	+8	+4 + video sessions.
16,000	+8	+8	+8 + video sessions.

Figure 20: BigFix Remote Control Broker Capacity Planning Requirements

Component	CPU	Memory (GB)	Storage (GB)
Controller	+1	+4	+1
Target	+1	+2	+2 + video sessions.
Gateway	+1	+2	+1

Figure 21: BigFix Remote Control Controller/Target/Gateway Capacity Planning Requirements

4.11 BigFix Operating System Deployment (OSD)

BigFix Operating System Deployment (OSD) provides an imaging service. It is an extension of the BigFix platform, meaning based capacity planning derives from the platform root server, database server, and relay infrastructure previously described. However, given OSD is distributing images that can be several gigabytes in size, ensuring the configuration is proper is critical. We will describe the key configuration points, generally based on guidance in the BigFix Non-Functional Requirements (NFR) guide (see the References section).

BigFix OSD key configuration points.

- A properly configured OSD service may provision up to 32 laptops per hour with a basic image in the range of 2GB to 8GB. The larger the image size, the more pressure on the system. For example, advanced images in the [20GB,30GB] range are not uncommon and will have an impact on the infrastructure and deployment times versus smaller images.
    - The maximum number of recommended concurrent deployments for a standard configuration is 45.
    - The maximum number of recommended concurrent deployments increases to 60 when multicast is enabled. More information on the multicast deployment impact is provided below.

- In all OSD scenarios, the target downloads the pre-installation environment (Windows WinPE or Linux Engine) from the TFTP bare metal server, downloads the OS image, and then installs the OS image. The source of the OS image varies depending on the OSD configuration and determines the infrastructure workload and requirements. The three options for the source of the OS image are as follows.
    - Windows Unicast: The target downloads the OS Image from the BigFix relay.
    - Windows Multicast: The target downloads the OS Image from the bare metal server.
    - Linux HTTP: The target downloads the OS Image from the bare metal server.
- In terms of deployment strategy, when aiming to increase the number of deployments within a specific timeframe, the greatest overload occurs during the initial phase until the OS image download is complete on the target. After this time, the target completes the remaining phases autonomously without impacting the other deployments. The suggested strategy is to launch a smaller group of targets initially and then start a new group once the first group has completed the OS image download phase, versus launching many targets simultaneously and waiting for full completion.
    - For example, to provision 32 laptops over an hour, pushing at most eight (8) deployments every fifteen (15) minutes may prove less impactful to the infrastructure versus “all at once”.
- The Bare Metal OS Deployment Server (aka the Preboot Execution Environment Server aka PXE Server) and/or network shares should be in close network proximity to the targets. See item PRF-INF-08 in the BigFix NFR guide for further detail.
- Network speeds throughout the deployment infrastructure are critical. For example, an 8GB image pushed to 32 laptops over an hour implies an aggregate network requirement of 18Mbps. This is in addition to existing bandwidth requirements. As always, data center speeds of 100Mb/s and up are recommended, and the laws of physics apply.
- The download cache throughout the infrastructure must be configured properly. The default is 1GB, and the recommendation is to configure in the interval [100GB, 300GB]. See item PRF-BIG-47 in the BigFix NFR guide for further detail.
- To monitor deployments, the following resources are available.
    - The Health Check Dashboard: URL
    - The Deployments Activity Dashboard: URL
- A detailed blog with screenshots for OSD is available here: URL. Note the blog recommendations for the download cache are considered too modest, and the recommendations above should prevail.
- Performance benchmarks for OSD demonstrate the following.
    - The concurrency impact is generally a 1% increase in duration per concurrent request for a well-tuned infrastructure.
    - Multicast offers benefits for concurrency via shared bandwidth. The net effect is reduced network bandwidth requirements with increased download times. The following examples apply for unicast and multicast on a 1Gbps network for a base Linux image over 5 targets.
        - Unicast: 5.5GB of traffic for a duration of 2 minutes.
        - Multicast: 2GB of traffic for a duration of 7 minutes.
    - Whether unicast or multicast is preferred is infrastructure dependent: in a low bandwidth environment, enable multicast.

4.12 BigFix Server Automation (SA)

BigFix Server Automation (SA) provides a fault tolerant solution for automation at scale. The capacity planning recommendations are for the SA versions 9.5.72 and above supporting the multi-node plan engine. SA is comprised of two (2) components.

- The Automation Plan Distribution Service.
  This is collocated with the BigFix root server and distributes work to the Plan Engine(s).
- The Automation Plan Engine(s).
  The engine(s) that executes the plans.

See the following figure for a sample multi-node deployment architecture. Some notes on the deployment architecture.

- The distribution service is typically accommodated by the root server capacity planning. When enabling SA, monitoring should be performed on the root server to verify resources are sufficient.
- Multiple Plan Engines should be deployed for redundancy and scalability.
- Each Plan Engine instance will utilize one half of the cores for the compute instance it is deployed on. If Fewer than eight (8) cores are available, only a single core will be consumed.

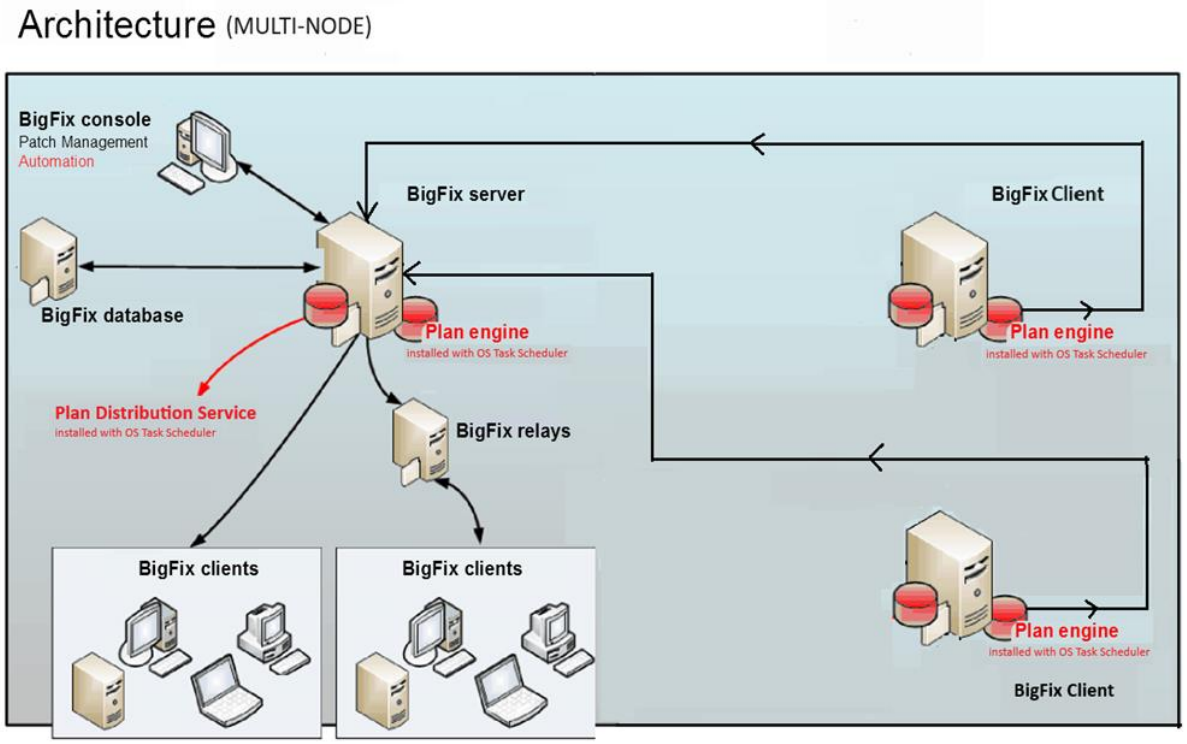


Figure 22: BigFix Server Automation Infrastructure

Plan Engine Instances	vCPU per Plan Engine	Memory (GB) per Plan Engine	Storage (GB) per Plan Engine
Minimum: 1 Recommended: 2 Maximum: Based on Workload	+ ≥ 8	+8	+1

Figure 23: BigFix Server Automation Plan Engine Capacity Planning Requirements

4.13 BigFix Compliance Analytics

BigFix Compliance Analytics provides reporting across three security domains: Security Configuration Reporting, Patch Reporting, and Vulnerability Reporting. It consists of an application server and a database server, with the combined capacity planning requirements provided in the following table. For the database server, the storage requirements may be a combination of the “Base” and “P&V” calculations provided. Additional configuration notes are provided after the storage calculations.

Agent Deployment Size	CPU [App Server, DB Server]	Memory (GB) [App Server, DB Server]	Storage (GB) [App Server, DB Server]
< 5,000	[4, 4]	[8, 8]	[50, 50GB + P&V]
10,000	[4, 4]	[8, 16]	[50, 50GB + P&V]
25,000	[4, 4]	[8, 16]	[50, 75GB + P&V]
50,000	[6, 6]	[16, 32]	[50, 165GB + P&V]
100,000	[8, 8]	[24, 32]	[50, Base + P&V]
300,000	[8, 8]	[32, 64]	[50, Base + P&V]

Figure 24: BigFix Compliance Analytics Capacity Planning Requirements

Storage notes.

- Base Storage Calculation.
  The base calculation applies in general to Compliance Analytics deployments.

  Base = 165GB + 1.5GB for every 1,000 endpoints.

  Further detail on the base storage requirements is available in the product reference: URL.
- P&V Storage Calculation.
  This calculation applies if Patch & Vulnerability reporting is configured and should be added to the base calculation.

  P&V = Reports Per Day x Pruning Threshold in Days x 1.5 x 25GB

  The P&V storage calculation is per report. Data pruning should be enabled based on the P&V storage calculation.

Additional configuration notes.

- The application and database servers should ideally be anti-collocated. This permits resource isolation with typically gains in storage IOPS. In the event it is desired to collocate the servers, system monitoring of resource health is recommended. Further information on monitoring resource health is available in the BigFix Non-Functional Requirements (NFR) guide (see the References section).
- The BigFix Compliance SQL Server Agent index maintenance job should be enabled for the database. The job runs under the SQLAgent service and is based upon the "BFEnterprise Full Database Index Reorganization" script. Third party scripts may optionally be used as long as they provide the required maintenance with minimal impact.
- In the event index rebuilds become problematic, one approach can be to take a maintenance outage, followed by dropping and recreating the index. This typically requires DBA oversight. The tem_analytics.pr.computer_group_patch_rollups PK index is one example of an index that requires large scale rebuilds.

4.14 Message Level Encryption (MLE)

Message Level Encryption (MLE) provides data encryption support for clients and is particularly valuable for insecure networks or when secure communication in general is required. It is worth noting MLE does not affect actions taken from the console or Fixes/Fixlets that are already protected by digital signatures. More information on MLE is provided in the References section.

The following table describes the capacity planning requirements for Message Level Encryption.

Component	Additional CPU	Additional Memory (GB)	Additional Storage (GB)
Message Level Encryption	+2	+4	n/a

Figure 25: Message Level Encryption Capacity Planning Requirements

5 Performance Management

Capacity planning and performance management go hand in hand. There is no standard workload for BigFix. Every enterprise has different requirements, infrastructure, and customization. This section will build upon the base capacity planning recommendations and offer a set of defined decision points for building an optimal BigFix deployment based on the business needs.

We will first describe some general performance management approaches. We will describe several management and configuration recommendations at the BigFix, database, operating system, and hypervisor levels. Database maintenance recommendations are provided in the separate maintenance guide (see the References section).

5.1 Performance Management Approaches

Several generic approaches will be described, ranging from ease of management through a decision tree for managing performance.

5.1.1 Performance vs Ease of Management

- In pure performance terms, the ideal environment is a physical deployment with multiple distinct NVME based flash drives. This offers maximum efficiency and parallelism, and scales in a deterministic way.
- The alternative approach is for ease of management: a virtual deployment with remote storage subsystems, and pooled database management services. This offers maximum density, and easily partitions across multiple teams within the business.
- Every installation needs to assess these trade-offs, and hybrid approaches are often used. It is possible to combine performance and ease of management, based on understanding and managing the overhead associated with ease of management.

5.1.2 The Storage Revolution

- When a system is perceived as running slow, the first reaction is to throw more processor cores at the problem. However, in approximately 80% of cases, the limiting factor is the storage subsystem.
- While improvements in processor capability have made headlines for decades, the most exciting improvements have been in storage.
- Most are aware of the improvements in flash-based storage, leaving behind solutions that required spinning magnetic platters with moving heads. However, the interfaces associated with the storage improvements have improved dramatically.
- Interfaces have migrated from IDE to AHCI to, at present, NVME. The NVME interface offers greater parallelism with reduced application and system load.
- The best thing you can do for your BigFix deployment is implement NVME flash storage.

5.1.3 When to Throw Hardware at the Problem

We will comment on simple approaches of “throwing hardware” at the problem.

- CPU:
  The first temptation is to throw CPU at the problem. On a physical system, improvements in the number of cores or clock speed can drive improvements. However, on a virtual system, allocating more cores than needed can actually put pressure on the hypervisor scheduler and degrade performance. CPU “upgrades” should be done carefully with awareness of the base problem.

- Storage:
  Storage improvements are always beneficial. Whether adding device paths, or upgrading storage, it is a beneficial move. However, storage should always be benchmarked. Just because a solution looks good on paper, does not mean it is configured and deployed correctly.
- Network:
  Network resource improvements are always beneficial, though often difficult to manage beyond the data center.
- Memory:
  Modern systems will generally use all the memory you can throw at them. If you have it, use it.

5.1.4 Collocation and Anti-collocation Options

As stated, the ideal server implementation is a single node physical instance. However, there can be good motivation for not collocating components. The ultimate decision is typically determined by organization structure and business needs.

- A general benefit of collocation in physical deployments is database communication paths tend to be optimal and advantages like shared memory may be exploited.
- A general benefit of anti-collocation in virtual deployments is smaller and easier to manage virtual machines may be deployed.
- The database server: A primary benefit of anti-collocation is being placed in a database administrator (DBA) managed pool with monitoring, backup support, etc.
- The WebUI server: there is no inherent advantage in running a standalone WebUI server, beyond the general benefits already described.
- Web Reports and BES Explorer: For these components, it can be advantageous to run multiple standalone instances for scalability and resource isolation.

5.2 BigFix Management Recommendations

We will provide configuration recommendations for the BigFix components.

5.2.1 BigFix FillDB Configuration

- The FillDB service has a “database boost” parameter that is key for maximum throughput.
  - For Windows, the boost is enabled by default for BigFix 9.5.10 onwards.
  - For Linux, the boost is enabled by default for BigFix 9.5 onwards.
  - For all prior releases, please consult the version specific product documentation.
- In the BigFix 9.5.5 release, a parallel FillDB implementation was delivered. The following parameters are managed, with the first set being for the base FillDB capability, and the second set being for the BigFix Query result processing capability that is also managed by FillDB.
  - ParallelismEnabled
    NumberOfParsingThreads
    NumberOfDBUpdatingThreads
  - ParallelismEnabledForQuery
    NumberOfParsingThreadsForQuery
    NumberOfDBUpdatingThreadsForQuery
  - The parallelism is enabled by default by the BigFix installer based on the hardware capability (number of cores). For example, there is a base report concurrency of [3

readers, 3 writers] if the machine has 6+ cores. If the machine has 10+ cores, the BigFix Query report concurrency is also set to [3, 3].

- When the base parallelism is enabled, FillDB throughput doubles with a nominal increase of 1 to 1.5 cores. Similarly, for BigFix Query, the parallelism can essentially cut the processing time for large result sets in half. Both of these are significant improvements, with no modification required by the user.
- It is possible in an environment with significant available system capability, that more parallelism may be enabled by the user with additional throughput improvements. However, caution should be used. The default parallelism provides an excellent combination of high throughput with low resource impact.
- The following figure provides an example of parallelism improvements. Here modest improvements in parallelism are in evidence as we proceed beyond the default of "3/3". However, once you go beyond "10/10", degradation is in evidence.

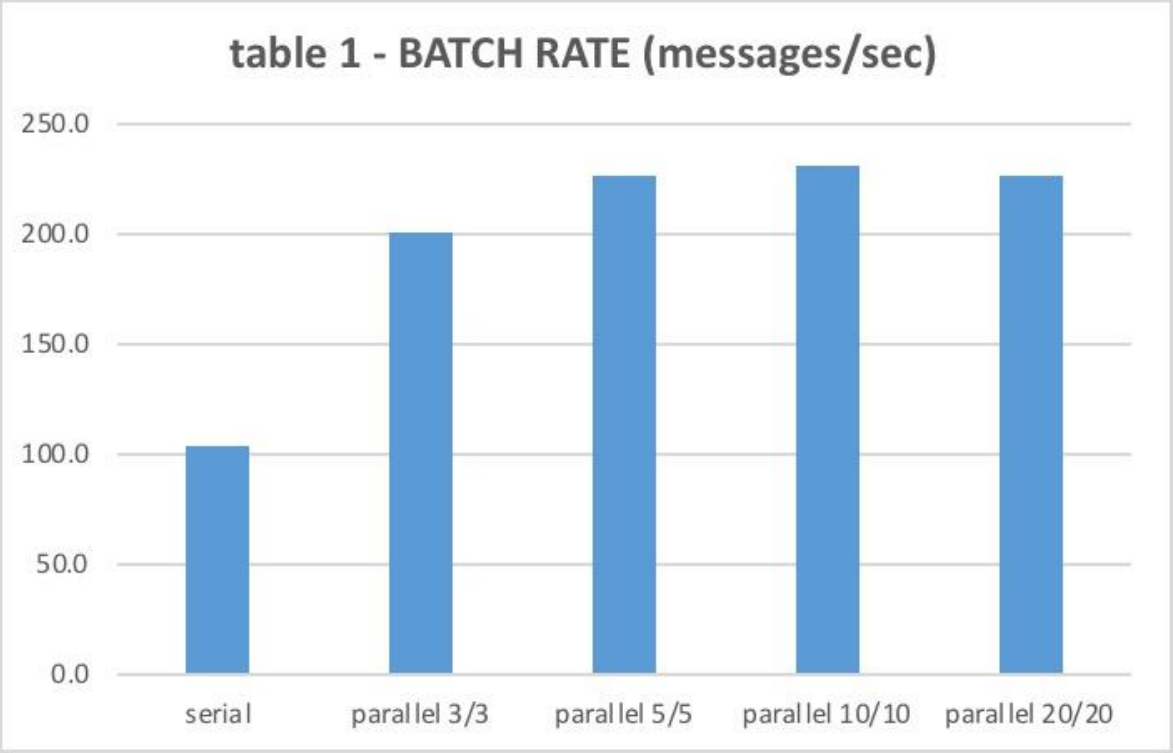


Figure 26: FillDB Parallelism Impact

5.2.2 BigFix Relay Configuration

- The _BESRelay_HTTPServer_MaxConnections configuration parameter will determine the maximum number of concurrent connections for the relay. This defaults to 2048 and may be changed for extreme high concurrency scenarios.
- For Windows it is possible that altering the TCP/IP configuration may be beneficial. See section 5.5.1 for more information on Windows TCP/IP management.

5.2.3 Agent Heartbeat Frequency

- The agent sends a heartbeat to the server (essentially, reporting in), every 15 minutes. As a BigFix deployment scales, the heartbeat activity can become significant. For example, if 250,000 agents are reporting every 15 minutes, that is over 278 heartbeats per second!
- In order to mitigate this, the general recommendation is to set the heartbeat interval to 15 minutes for every 10,000 agents. For example, for a 250,000 agent deployment, this would mean a heartbeat on the order of 6 hours. At this setting, there would be on the order of 12

heartbeats a second. This is much more manageable.

- The heartbeat may be set in the BigFix console preference section.

5.2.4 Agent Multicore Exploitation

- The BigFix Agent has a continuously increasing content stream, with no end in sight. This is being amplified as Artificial Intelligence drives Security Operations. In addition, many systems, especially non server systems, have available processing power. Given these conditions, BigFix 11.0.7 has introduced agent multicore exploitation.
- The basic concept is the agent will dynamically scale to the core allocation on startup. This is enabled via three new client settings (consult the knowledge center for detailed information):
  _BESClient_Resource_ScaleWorkTimeWithCPUCores
  _BESClient_Resource_MaxCoreUtilization
  _BESClient_Resource_WorkTypeScalar
- It is recommended to enable this feature on all non-performance critical systems with the default configuration values. It will ensure more work gets done per unit of time and accelerate agent operations.
- One common misconception is the agent workload will be evenly distributed across available cores. This is up to the CPU scheduler. A better way to think of this is the aggregate CPU resources will be better utilized and distributed over time.
- A basic mathematical view would be on a, say four (4) core system the agent workload will run four (4) times faster. It is not quite this simple, as there are serialization effects, reduced context switching, etc. As an example, some reference workloads run 3,200% faster on a four (4) core deployment. This feature offers a simple, fundamental way to do more with BigFix.

5.2.5 Console Refresh Frequency

- The BigFix console refresh frequency determines the interval for populating (refreshing) the console cache. The default value is 15 seconds, meaning every 15 seconds there is a database workload (impacting CPU, disk, network, etc.) in order to maintain the cache.
- Setting the refresh frequency is a trade-off between data currency and system workload. The setting is ultimately derived from the number of console operators, the size of the estate, and the desired user experience. For a large estate with many concurrent users, a refresh frequency of 300 seconds is not out of the question.
- The console refresh frequency may be set in the BigFix console preference section.

5.2.6 Data Archiving

A variety of data archiving approaches are available and should be performed at regular intervals.

- Deletion of closed and expired actions.
  This is a console activity that will remove the actions from the console view. While the action will still persist in the database, this cleanup approach will reduce the console workload.
- Computer removal.
  A computer removal utility (URL) is available to remove obsolete computers and thereby reduce the overhead for database operations.
- Audit cleanup.
  An audit cleanup utility (URL) is available to prune entries based on a variety of criteria, including time. The audit cleanup should be done in accordance with the enterprise audit policies. For example, a database archive may be generated to store the audit content for that point in time, with the subsequent cleanup serving to reduce the overhead for database operations.
- The BigFix 9.5 release offers significant improvements for data archiving and management.

- Database cleanup tools.
  You may use the BESAdmin interface on Windows systems or the BESAdmin command line on Windows and Linux systems to remove data about computers, custom Fixlets, properties, analyses, and actions and to update the PropertyIDMap table with changes.
- FillDB log rotation.
  The log rotation feature is active by default with LogFileSizeLimit set to 100 MB.

5.2.7 WebUI

The WebUI is configured to perform well “out of the box”. Some recommended guidelines follow.

- Improvements in BigFix 9.5.10 provide a superior base for the WebUI deployment, with improved maintenance operations and transaction concurrency management. As a result, the WebUI should be deployed on a minimum of a 9.5.10 base.
- The WebUI employs significant caching and queue management. For example, a number of tables under the WebUI schema will be in evidence in the BigFix Enterprise database. There is also a stored procedure queue workload manager governed via a Stored Procedure Queue Concurrency (SPQC) setting for the WebUI. This setting is pre-configured and should not be adjusted without deep understanding. For example, reducing the setting will result in longer refresh times, while increasing the setting will impact concurrency and locking.
- For best performance, the recommended MS SQL configuration settings should be applied (see the next section).

5.2.8 The Plugin Portal

The Plugin Portal involves relevance evaluation. The concurrency of the relevance evaluation may be customized through a configuration option with direct benefits for portal throughput. Some notes on this option follow.

- The configuration option is:

  Windows (via the Windows registry):
  HKEY_LOCAL_MACHINE\SOFTWARE\WOW6432Node\BigFix\EnterpriseClient\Settings\Client_BESPluginPortal_Performance_ThreadLimit

  Linux (via client setting): BESPluginPortal_Performance_ThreadLimit
- The default threading value for relevance evaluation is automatically set to the number of cores available. The maximum allowed value is 128.
- Given the configuration is automatically managed for new deployments in BigFix 10.0.4, it is recommended legacy deployments remove this setting so it may be automatically managed.
- The general recommendation applies to monitor performance before and after a configuration change.

The BigFix Plugin Portal capacity planning is in terms of the number of cloud devices deployed, with implications for the completion of the loop cycle time, depending on the number of operators and devices assigned. For more advanced Plugin Portal optimization of the loop cycle time, the following steps may be followed.

- The administrator should monitor the loop cycle time and check if it is safely contained in the default refresh interval by enabling the verbose log for the Plugin Portal instance:
  _BESPluginPortal_Log_EnabledLogs -> timing
  _BESPluginPortal_Log_Verbose -> 1
- Then monitor the Plugin Portal log file (BESPluginPortal.log) content and filter for the elapsed time, looking for the row : Exiting RunOneLoopIteration (<***elapsed time***> ms).
- In the event the administrator would like to change the interval for the loop cycle time, the

following property may be adjusted:
"Update <*VMware* | *Microsoft Azure* | *Google Cloud* | *Amazon Web Services*> Plugin Refresh Interval", updating the value: "Refresh Interval in Minutes:"

- For example if the loop cycle time exceeds the default two (2) hours, then run the task "Update <*VMware* | *Microsoft Azure* | *Google Cloud* | *Amazon Web Services*> Plugin Refresh Interval", updating the value: "Refresh Interval in Minutes:" to a value greater than two (2) hours. For example, set the value to three (3) hours and restart the services.

5.2.9 The MDM Docker Host

The following configuration recommendations have been used for stress testing of the Docker host instance.

- /etc/security/limits.conf
  - root soft nofile 100000
  - root hard nofile 100000
  - root soft nproc 100000
- /etc/sysctl.conf
  - net.core.rmem_default=1000000
  - net.core.wmem_default=1000000
  - net.core.rmem_max=1000000
  - net.core.wmem_max=1000000
  - net.ipv4.tcp_rmem=4096 87380 167177216
  - net.ipv4.tcp_wmem=4096 65536 167177216

5.3 MS SQL Management Recommendations

We will describe management options for MS SQL, including parallelism and query performance.

5.3.1 MS SQL Parallelism

A database management system is designed to drive high parallelism on modern day multiprocessor hardware. However, parallelism typically has a threshold where throughput levels off, and may even degrade. The default MS SQL configuration is typically enabled for maximum parallelism, which is not ideal. The BigFix 10.0 release does automatic management of the following parallelism parameters upon upgrade. Changes should only be made with expert insight and monitoring.

- Maximum Degree of Parallelism (MAXDOP).
  The Maximum Degree of Parallelism determines how many processors are utilized for a parallel plan. The default value is zero (0), which essentially means "unlimited". The proper value needs to be derived from the specific physical hardware of the database server and is typically a low value in the interval of [4,8].
  - The specific hardware parameter is the number of NUMA cores. This may be derived by querying the MS SQL table sys.dm_os_memory_nodes. The returned value will be the number of nodes + 1.
  - Information on how to set the MAXDOP and an associated calculator are provided at this location: URL. The link is also provided in the References section.
- Cost Threshold for Parallelism (CTFP).
  The cost threshold for parallelism determines the level where SQL server will build and execute parallel plans for queries. The default value is five (5). The recommended value is

fifty (50).

Further details on configuring these values are provided in the References section.

5.3.2 MS SQL Query Performance

MS SQL query performance can be inspected via the package cache. The package cache is the set of currently active compiled statements. It may be derived via the following query. Note the query "order by" clause may be modified to determine the sorting of the result set. The results are ideally reviewed by an expert, but even a lay person can derive conclusions based on query times, lock impact, etc.

```
SELECT TOP 100 SUBSTRING(qt.TEXT, (qs.statement_start_offset/2)+1,
((CASE qs.statement_end_offset
WHEN -1 THEN DATALENGTH(qt.TEXT)
ELSE qs.statement_end_offset
END - qs.statement_start_offset)/2)+1) AS query_text,
qs.execution_count,
qs.total_rows, qs.last_rows, qs.total_rows / qs.execution_count avg_rows,
qs.total_dop, qs.last_dop, qs.total_dop / qs.execution_count avg_dop,
qs.total_logical_reads, qs.last_logical_reads,
qs.total_logical_writes, qs.last_logical_writes,
qs.total_physical_reads, qs.last_physical_reads,
qs.total_worker_time, qs.last_worker_time,
qs.total_worker_time / qs.execution_count avg_worker_time,
qs.total_elapsed_time / 1000 total_elapsed_time_in_ms,
qs.last_elapsed_time / 1000 last_elapsed_time_in_ms,
qs.total_elapsed_time / qs.execution_count / 1000 avg_elapsed_time_in_ms,
qs.creation_time, qs.last_execution_time, qp.query_plan
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
ORDER BY avg_elapsed_time_in_ms DESC -- CPU avg
```

Figure 27: MS SQL Query Performance Package Cache

5.3.3 MS SQL Always On Availability Groups for Read Scale

An MS SQL Always On Availability Group (hereafter referred to as an MS SQL AG) may be deployed for high availability of the BigFix databases. It may also be used for "read scale". For example, you may have the following deployment.

- The BigFix primary database BFEnterprise.
- A read only replica of the BFEnterprise database.
- An off-site disaster recovery replica of the BFEnterprise database.

What is read scale? Essentially the deployment of a read only replica for the secondary provides a horizontal scalability solution where "read only" applications may have a dedicated server semi-independent of the primary. A good example where this is valuable is as an Extract-Transform-Load (ETL) data source for BigFix Insights. Custom integrations may also benefit. As of this writing, it is not recommended for the BigFix Inventory (BFI), Security & Compliance Analytics (SCA), or WebUI applications; these applications should always connect to the primary database.

Deployment of the MS SQL AG is relatively transparent to the application. The MS SQL AG may generally be enabled for BigFix upgrade operations. Care should be taken to ensure the storage and query performance is maintained across the cluster. In addition, it is recommended to have dedicated

network adapters between the nodes of the group. The MS SQL query performance approach in the previous section offers a simple way to monitor.

Information on configuring the listener for an MS SQL AG is available here: URL.

5.3.4 MS SQL Cloud Services

There are several cloud services options for MS SQL. For example, Google Cloud SQL for SQL Server. The official platform documentation provides information on the specific offerings supported.

In terms of performance management of the cloud services, they will perform acceptably if the following considerations are met.

- Network latency from the application to the cloud service must be less than 1ms. For example, an on-premises deployment of the application with a remote cloud service is generally not recommended due to latency concerns.
- Suitable vCPU, RAM, and storage allocation per the capacity planning recommendations in this guide.
- Storage class selection that meets the minimum requirement of 5K IOPS @ 1ms latency.

5.3.5 MS SQL Secure Settings

A set of MS SQL secure settings are available to provide additional security enforcement for database operations. The set of settings and their sample reference documentation follows.

- Force Encryption.
  The force encryption option for MS SQL ensures encrypted communication to the DBMS engine.
  Enablement is via MS SQL best practices: URL.
- Common Criteria.
  Common criteria compliance includes such dimensions as Residual Information Protection (RIP), login audit, and authorization changes.
  Enablement is via MS SQL best practices: URL.
- Service Protection.
  By default, the MS SQL instance is configured to support up to 32,767 connections. This has the potential to degrade the service to the point it may effectively yield a Denial of Service (DoS) attack. As a result, a practical limit of 1,000 is recommended.
  Enablement is via MS SQL best practices: URL.
- ODBC Strong Encryption.
  This option will enforce strong encryption for the ODBC connection. It is recommended if the database server is anti-collocated with the BigFix server(s).
  Enablement is via MS SQL best practices: URL.

These settings do have run time impact as more work needs to be done for database operations. Diligent monitoring before and after the configuration changes is recommended. Observation on the aggregate impact of the above changes for a reference deployment follow. The impact is considered manageable, but non-trivial.

- Root server impact of 20% additional processor time.
- FillDB service impact of 5% additional processor time, with 20% throughput reduction.
- GatherDB service impact of 100% additional processor time, though this is a short duration low cost service (e.g., single core utilization levels).

5.3.6 MS SQL Transparent Data Encryption

MS SQL Transparent Data Encryption (TDE) encrypts data at rest. This is a security practice to ensure if the physical storage device is compromised the data will be inaccessible.

Pages are encrypted when written to disk, and decrypted when read into memory. As indicated by the feature name, this is transparent to the application. There is additional processing overhead for encryption management at scale. Encryption operations are typically managed asynchronously. Processor utilization should be monitored before and after the change. The size of the database does not change with TDE.

The features and limitations of TDE are described here: URL

5.4 DB2 Management Recommendations

We will describe DB2 instance configuration and query performance.

5.4.1 DB2 Instance Configuration

The DB2 instance configuration applies to all databases running beneath it. The following configuration parameters are typically set for the instance by BigFix, and may not be present if the instance is a custom build, migrated, etc. It should be verified they are enabled and, if they are not, enable them.

```
db2set DB2_SKIPINSERTED=ON
db2set DB2_EVALUNCOMMITTED=YES
db2set DB2_EXTENDED_OPTIMIZATION=ON
db2set DB2_SKIPDELETED=ON
db2set DB2COMM=TCPIP
db2set DB2AUTOSTART=YES
```

Figure 28: DB2 Instance Configuration

5.4.2 DB2 Query Performance

A basic mechanism to manage DB2 query performance is via snapshots. The snapshot provides a point in time view of all transactions on the system. The following steps may be used to derive a snapshot. The results are ideally reviewed by an expert, but even a lay person can derive conclusions based on query times, lock impact, etc.

```
Step 1: db2 update dbm cfg using DFT_MON_BUFPOOL on DFT_MON_LOCK on DFT_MON_SORT
on DFT_MON_STMT on DFT_MON_TIMESTAMP on DFT_MON_TABLE on DFT_MON_UOW on
Step 2: Reset the monitor switches: db2 reset monitor all
Step 3: Run the workload.
Step 4: Get the dbm snapshot: db2 get snapshot for dbm > dbm.txt
Step 5: Get the database snapshots: db2 get snapshot for all on <db> >
<db>.txt  (substitute <db> and iterate over all databases).
Step 6: db2 update dbm cfg using DFT_MON_BUFPOOL off DFT_MON_LOCK off
DFT_MON_SORT off DFT_MON_STMT off DFT_MON_TIMESTAMP on DFT_MON_TABLE off
DFT_MON_UOW off
```

Figure 29: DB2 Query Performance Snapshot

5.5 Operating System Management Recommendations

Both Linux and Windows are evolved and refined operating systems that can be used as the base for high performance systems. In terms of operating system tuning, very little tuning is typically required.

However, specific guidelines for CPU, memory, IO, and network management apply.

- CPU and memory are very straight ahead, with capacity planning and monitoring approaches provided in this paper, along with recommendations for virtual deployments.
- IO and network management are more complex. Fortunately, there are some excellent BigFix Knowledge Base articles addressing this. Rather than duplicate this content here, please consult the References section.
- We will describe some special areas of operating system management: Windows port management, Linux “swappiness”, and the Linux “ulimit”.

5.5.1 Windows TCP/IP Port Management

- In the event that TCP/IP ports appear to be exhausted, configuration changes may improve the operational behavior. For example, if the BigFix logs contain a number of HTTP 28 timeout messages, it is possible that port exhaustion is the cause.
- The base configuration is to modify the Windows registry for reducing the “time wait” interval used to recycle the TCP/IP port resources. To be more specific, it can be useful to reduce the client TCP/IP socket connection timeout value from the default value of 240 seconds, to the minimum lower bound of 30 seconds.
- More information, including how to manage the specific registry update, is available in the BigFix NFR Guide (Item PRF-INF-17) and the following technical note: URL.

5.5.2 Linux Swappiness

- The Linux swappiness kernel parameter (vm.swappiness) is a value in the interval [0,100] that defines the relative weight of swapping out runtime memory, versus dropping pages from the system page cache. The default value is typically 60.
- The recommendations for setting this value are as follows.
    - For a dedicated database management server, the swappiness should be set to zero (0).
    - For a database management server collocated with the BigFix application server, the swappiness should be set to ten (10).
- Further details on managing DB2 performance are provided in the References section.

5.5.3 Linux ulimit Management

For Linux operating system defines a system ulimit for the maximum number of open files allowed for a process (i.e., the nofiles option when you run the command “ulimit –a”). For relays see Section 4.7.1.3. For the DB2 instance, the value for this kernel limit should be either “unlimited” or “65536”.

5.5.4 Virtualization Management

In today’s modern enterprise, virtualization is seen as a powerful way to address the management of cost and scale. In general terms, performance management of physical servers tends to be simpler. Resources are isolated, there is no hypervisor involved, and the operating system view of performance is a direct indicator of system and application performance.

In order to simplify performance management and keep latency characteristics to a minimum, the first recommendation is always to deploy on physical hardware. However, it is still possible that a virtual deployment is still desired (whether enterprise standards, high skill levels in the team for virtual system performance, etc.). To manage BigFix in a virtual environment, precautions must be taken to ensure performance. We will describe some of the key management aspects. We will then reinforce the fact that monitoring and understanding is critical in a virtual world.

5.5.5 "Right Sizing" the CPU Allocation

- When deploying a physical server, additional CPU resources are generally seen as passive, or perhaps even beneficial. In a virtual environment, an oversized VM may actually degrade performance. The reason is in a shared deployment model, larger VMs require greater scheduling and orchestration effort. This may lead to scheduling delays or wait time. As a result, "right sizing" is critical.
- The capacity recommendations in this paper are the starting point, with monitoring being essential. Some classic elements for monitoring follow.
    - CPU ready.
      This is the percentage of time the VM is ready to be run but is waiting due to scheduler constraints.
    - CPU wait.
      The amount of time the CPU spends in wait state.
- A set of VMware performance troubleshooting guides are provided in the References section.

5.5.6 VMware Snapshot Management

- Snapshots are a powerful tool in virtual environments. In addition, many teams new to virtualization start to leverage snapshots as a backup approach. In the context of VMware, it cannot be emphasized enough that snapshots should not be used for a backup policy!
- To understand why, it is strongly recommended to read the VMware literature in the References section of this paper. Essentially, snapshots result in the chaining of images with degradation incurred as a result of managing the chains. It is not unusual to see degradation on the order of hundreds of percent. To further compound the issue, such performance issues are often difficult to diagnose as they are obscured by the hypervisor.

5.5.7 VMware Latency Management

- With VMware vSphere 5.5, it is possible to set the latency sensitivity of a virtual machine. This serves to reduce the impact of virtualization with improved application performance, at the expense of "dedicated" resources. Further information is available through the VMware content in the References section.

5.5.8 Virtual IO Management

- Out of all system resources, IO is typically the most difficult to manage. High performance IO subsystems are relatively expensive, and prone to failure if redundancy is not managed. In addition, many solutions that perform well in a physical environment (say in the range of 5,000 to 10,000 IOPS) may plummet in a virtual environment (to, say, 100 IOPS).
- As a result, in any virtual environment it is critical to benchmark and monitor the IO subsystem. Some recommendations for a proper IO benchmark follow.
    - The workload pattern (block size, read %, write %, randomness) is important. The table below shows some recommended workload profiles. All should be exercised to demonstrate the IOPS of the available system.
    - As BigFix drives multiple IO "workers" so should the benchmark. A minimum of an eight (8) core system with eight (8) workers is recommended. Note more workers can lead to degradation and determining the parallelism threshold where degradation occurs can be beneficial (and, obviously, the higher this threshold is the better).
    - A recommended tool for the benchmark is Iometer (see the References section).

Workload	Block Size	Read %	Random %
Workload 1: Stock	4KB	25%	0%
Workload 2: Open Action Profile	8KB	10%	20%
Workload 3: REST API Profile	8KB	90%	20%

Figure 30: IO Benchmark Workload Profiles

5.5.9 The Linux IO Scheduler

- Each Linux instance has an IO scheduler. The intent of the IO scheduler is to optimize IO performance, potentially by clustering or sequencing requests to reduce the physical impact of IO.
- In a virtual world, however, the operating system is typically disassociated from the physical world through the hypervisor. As a result, it is recommended to alter the IO scheduler algorithm so that it is more efficient in a virtual deployment, with scheduling delegated to the hypervisor.
- The default scheduling algorithm is typically “cfq” (completely fair queuing)[3]. Alternative and recommended algorithms are “noop” and “deadline”. The “noop” algorithm, as expected, does as little as possible with a first in, first out queue. The “deadline” algorithm is more advanced, with priority queues and age as a scheduling consideration. System specific benchmarks should be used to determine which algorithm is superior for a given workload. The general recommendation is to use the “deadline” scheduler.
- The following console output shows how to display and modify the IO scheduler algorithm for a set of block devices. In the example, the “noop” scheduler algorithm is set. Note to ensure the scheduler configuration persists, it should be enforced via the operating system configuration (e.g. /etc/rc.local).

```
[root@deehtdb0a2ccxra ~]# echo noop > /sys/block/sda/queue/scheduler
[root@deehtdb0a2ccxra ~]# echo noop > /sys/block/sdb/queue/scheduler
[root@deehtdb0a2ccxra ~]# echo noop > /sys/block/sdc/queue/scheduler
[root@deehtdb0a2ccxra ~]# cat /sys/block/sdc/queue/scheduler
[noop] anticipatory deadline cfq
[root@deehtdb0a2ccxra ~]# cat /sys/block/sda/queue/scheduler
[noop] anticipatory deadline cfq
[root@deehtdb0a2ccxra ~]# cat /sys/block/sdb/queue/scheduler
[noop] anticipatory deadline cfq
```

Figure 31: Modifying the Linux IO Scheduler

[3] With Red Hat Enterprise Linux 7, the default scheduler has been set to “deadline”.

REFERENCES

BigFix Non-Functional Requirements: A Checklist Approach

BigFix Maintenance Guide

MX Performance Toolkit for BigFix

BigFix Knowledge Center

BigFix Resource Center

BigFix System Requirements

Iometer

NVMe SSDs: Everything you need to know about this insanely fast storage

MS SQL Maximum Degree of Parallelism

MS SQL Cost Threshold for Parallelism

Performance Best Practices for VMware vSphere™ 8.0

Best practices for virtual machine snapshots in the VMware environment

VMware: Troubleshooting ESX/ESXi virtual machine performance issues

VMware: Troubleshooting virtual machine performance issues

VMware: Performance Blog

Notices

This information was developed for products and services offered in the U.S.A.

HCL may not offer the products, services, or features discussed in this document in other countries. Consult your local HCL representative for information on the products and services currently available in your area. Any reference to an HCL product, program, or service is not intended to state or imply that only that HCL product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any HCL intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-HCL product, program, or service.

HCL may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to HCL TECHNOLOGIES LIMITED email: products-info@hcl.com

The following paragraph does not apply to the United Kingdom or any other country where such provisions are inconsistent with local law: HCL TECHNOLOGIES LIMITED PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some states do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. HCL may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-HCL Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this HCL product and use of those Web sites is at your own risk.

HCL may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Licensees of this program who wish to have information about it for the purpose of enabling: (i) the exchange of information between independently created programs and other programs (including this one) and (ii) the mutual use of the information which has been exchanged, should contact HCL TECHNOLOGIES LIMITED email: products-info@hcl.com

Such information may be available, subject to appropriate terms and conditions, including in some cases, payment of a fee.

The licensed program described in this document and all licensed material available for it are provided by HCL under terms of the HCL License Agreement or any equivalent agreement between us.

Any performance data contained herein was determined in a controlled environment. Therefore, the results obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

Information concerning non-HCL products was obtained from the suppliers of those products, their published announcements or other publicly available sources. HCL has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-HCL products. Questions on the capabilities of non-HCL products should be addressed to the suppliers of those products.

All statements regarding HCL's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

All HCL prices shown are HCL's suggested retail prices, are current and are subject to change without notice. Dealer prices may vary.

This information is for planning purposes only. The information herein is subject to change before the products described become available.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

If you are viewing this information softcopy, the photographs and color illustrations may not appear.

Trademarks

HCL, and other HCL graphics, logos, and service names including "hcltech.com" are trademarks of HCL. Except as specifically permitted herein, these Trademarks may not be used without the prior written permission from HCL. All other trademarks not owned by HCL that appear on this website are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by HCL.

IBM and other IBM graphics, logos, products and services are trademarks or registered trademarks of International Business Machines Corporation in the United States, other countries, or both. A current list of IBM trademarks is available on the Web at "Copyright and trademark information" at http://www.ibm.com/legal/copytrade.shtml.

Intel, Intel logo, Intel Inside, Intel Inside logo, Intel Centrino, Intel Centrino logo, Celeron, Intel Xeon, Intel SpeedStep, Itanium, and Pentium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

Oracle database, Java and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

VMware's and all VMWare trademarks and logos are trademarks or registered trademarks in the United States and certain other countries.

Dell, EMC, DellEMC and other trademarks are trademarks of Dell Inc. or its subsidiaries in the United States and certain other countries.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo and JBoss are registered trademarks of Red Hat, Inc. in the U.S. and other countries. Linux is a registered trademark of Linus Torvalds. All other trademarks are the property of their respective owners.

Mozilla and all Mozilla trademarks and logos are trademarks or registered trademarks in the United States and certain other countries.

Google LLC All rights reserved. Google and the Google Logo are registered trademarks of Google LLC.

NETAPP, the NETAPP logo, and the marks listed at www.netapp.com/TM are trademarks of NetApp, Inc.

www.hcltech.com

hello there! I am an Ideapreneur. i believe that sustainable business outcomes are driven by relationships nurtured through values like trust, transparency and flexibility. i respect the contract, but believe in going beyond through collaboration, applied innovation and new generation partnership models that put your interest above everything else. **R**ight now 119,000 ideapreneurs are in a **r**elationship Beyond the Contract™ with 500 customers in 32 countries. **how can I help you?**

Relationship™ BEYOND THE CONTRACT

HCL